OBJECT-ORIENTED ANALYSIS & DESIGN

Activity Diagrams

Modelling behaviour: from use cases to workflows to running code

Last time you ordered food for delivery, list every step from tapping “Order” to the doorbell.

Then circle the steps that...

(a) branch on a condition (b) happen at the same time

(c) hand work to another party (d) wait for something

Four UML concepts

- “If the restaurant is closed, pick another” → a DECISION node with GUARDS
- “Kitchen cooks while a courier is assigned” → FORK and JOIN
- “Restaurant / platform / courier each do their part” → PARTITIONS (swimlanes)
- “Courier waits until the food is boxed” → SYNCHRONIZATION at a join
-
- Activity diagrams give these four ideas a precise, standard notation.

Why model behaviour at all?

- No object exists in isolation: objects request services from other objects by sending messages (client-server view of computing)
- A system's USE CASES name the services it must provide, but a use case is text; we need a model of the FLOW of behaviour
- The behaviour model answers: what happens, in what order, what in parallel, what waits for what, who is responsible
- UML offers three behaviour views: activities (flow of actions), interactions (messages between objects), state machines (one object's lifecycle)
- Today: activity diagrams, the workflow view

Where activity diagrams live in the SDLC

- Requirements: use cases capture WHAT the system must do (text + use-case diagram)
- Analysis: the problem domain is refined, the behaviour model is built; activity diagrams are a primary analysis artifact
- Design: the same diagrams are refined toward implementation, partitions become classes/services, guards become conditions
- Implementation & test: diagrams become code paths and test scenarios
- In UML terms: an activity diagram is a BEHAVIOUR diagram

What is an activity diagram?

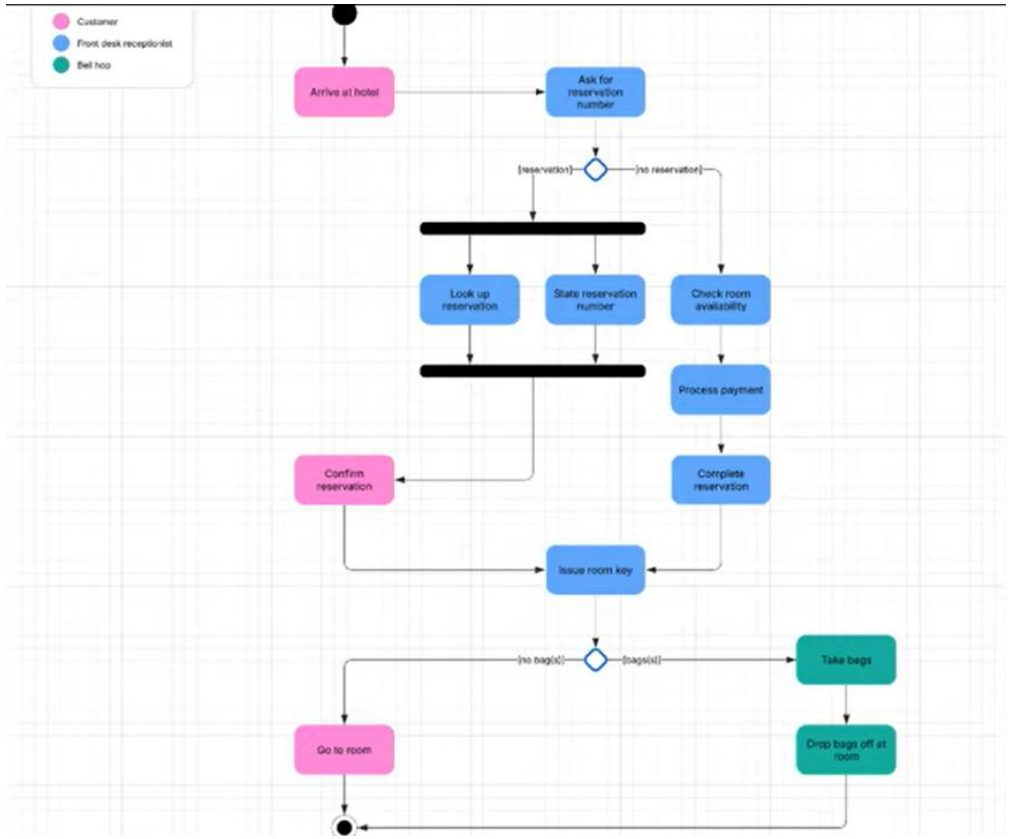
- Definition: a UML behaviour diagram that shows the flow of control and object flow between actions, with emphasis on sequence and conditions
- Like flowcharts, but with four superpowers:
  - true parallelism (fork / join), not just boxes and arrows
  - partitions (swimlanes): who performs each action
  - object flow: data moving between actions
  - formal token semantics: UML 2 defines execution as tokens flowing along edges, a formalism with roots in Petri nets
- Typical uses: business process modelling, workflow design, use-case realization, algorithm documentation

Activity diagrams vs flowchart

- Key technical differences

While flowcharts show simple linear processes, activity diagrams support complex logic

- Explicit parallel processing
- Multithread workflows
- Standardized UML notation
- Object flow tracking



Notation cheat sheet - keep this in your head

Activity diagram notation at a glance (UML 2.5)



Initial node
(flow starts here)



Action
(rounded rectangle)



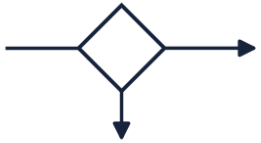
Control flow edge
(sequence)



Guard [condition] on edge
(evaluated at run time)



Object node
(data flowing)



Decision node
(pick ONE outgoing edge)



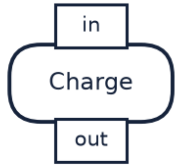
Merge node
(alternate flows reunite;
non-blocking)



Fork node
(split into concurrent flows)



Join node
(wait for ALL flows;
synchronizes)



Pins: input/output
object nodes of an action



Flow final node
(ends THIS flow only)



Activity final node
(ends EVERYTHING)



Data store
(keeps every token)



Connector
(readability / off-page)

Customer	System

Partitions (swimlanes):
who does what

Actions: the atoms of the flow

- An ACTION is a single atomic step within the activity, not decomposed further inside this diagram
- Named with a verb phrase; the name should survive reading aloud in a design review

GOOD names

Validate input

Process payment

Send confirmation email

Update account balance

POOR names

Do something

Process

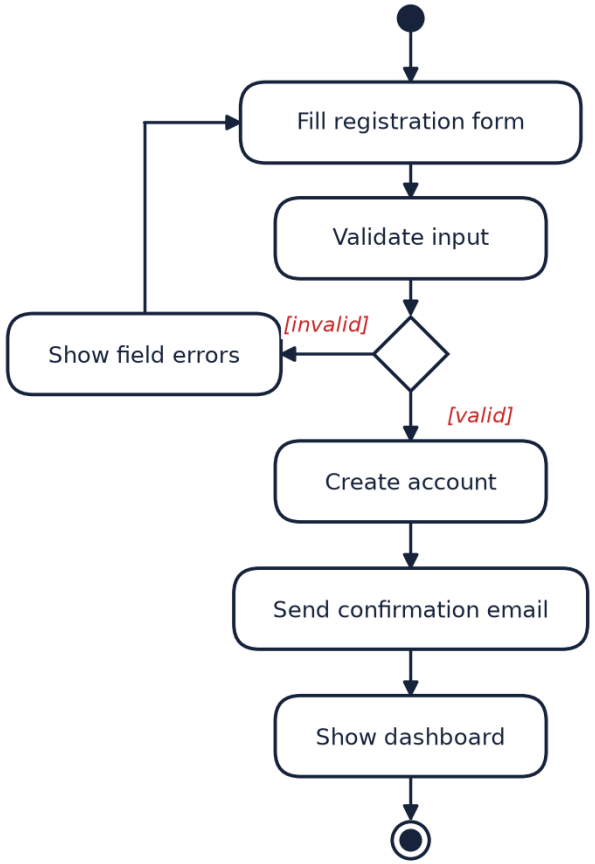
Handle

Manage system

Edges, guards, connectors

- Activity edge: directed connection along which control (or an object) flows, drawn as an arrow
- Guard: a condition in square brackets, evaluated at run time; the edge may be traversed only if it holds
- Guards leaving one decision node must be mutually exclusive, otherwise the choice is undefined
- Connector: a small named circle that continues an edge elsewhere on the page, pure readability, no semantics

First full diagram: account signup

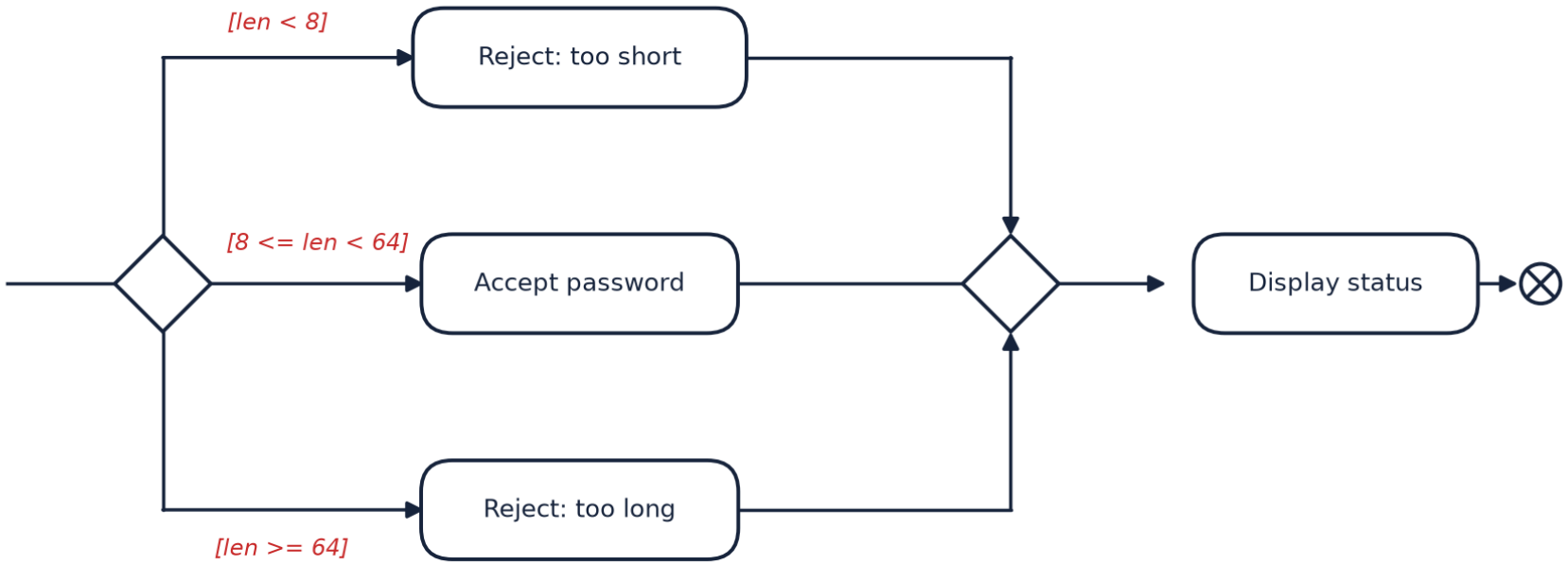


One loop, one decision: the smallest useful activity diagram.

Starts and stops

- Initial node (filled circle): where the flow starts when the activity is invoked, exactly one per activity
- Flow final node (circle + cross): terminates ONE flow; other concurrent flows keep running
- Activity final node (bullseye): stops EVERYTHING, all flows in the activity end
- Choosing wrong is a real bug: ending one branch with an activity final kills its parallel siblings

Decision & merge: the alternate-path pair

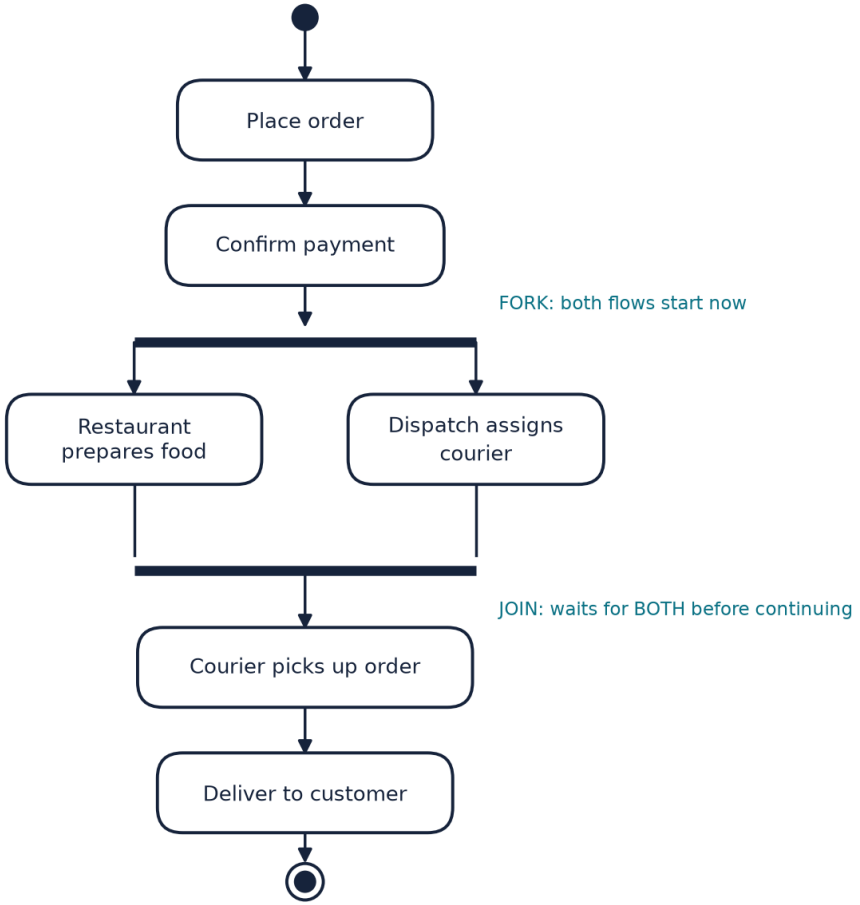


Decision: exactly one guarded path fires. Merge: the alternate paths reunite - no waiting, no synchronization.

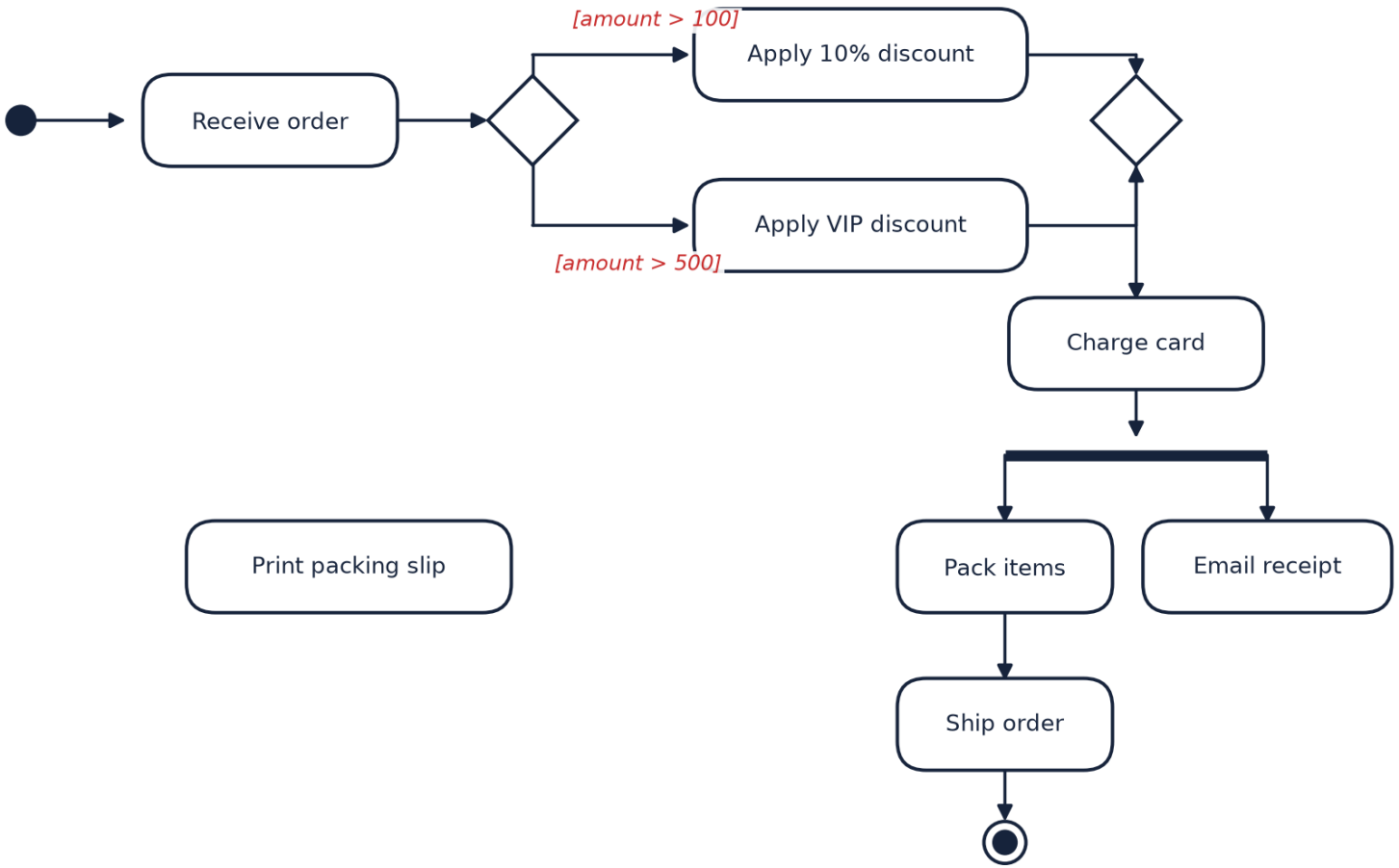
Concurrency is everywhere

- Concurrency: multiple tasks executing, interacting and collaborating in overlapping time
- Restaurant: several dishes cooked at once, one table served together
- Airport: many check-in counters, one flight departure that waits for all checks
- Your laptop: file downloads while you edit a document
- Software systems are concurrent by nature - client objects and server objects progress independently
- A flowchart cannot express this. An activity diagram can: fork and join.

Fork & join: the parallel pair



Bug hunt


[amount > 100]
Apply 10% discount
Receive order
Apply VIP discount
[amount > 500]
Charge card
Print packing slip
Pack items
Email receipt
Ship order

Partitions (swimlanes): who does what

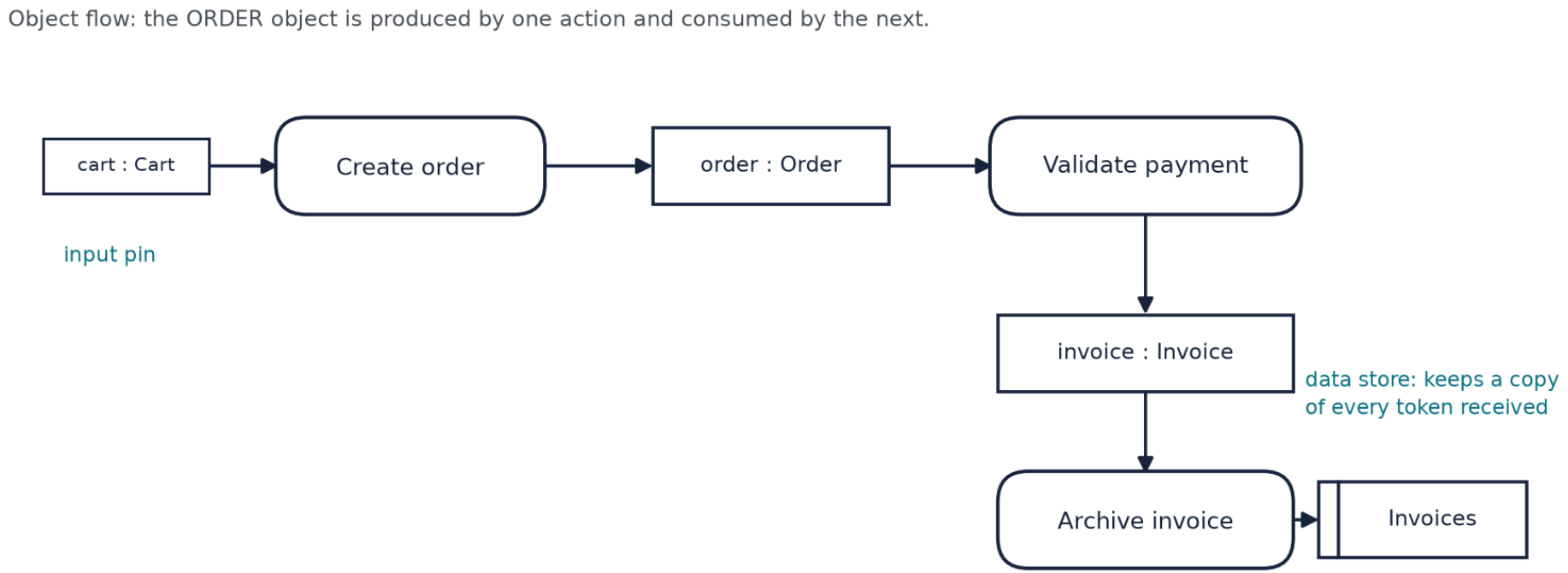
- An activity partition groups actions with a common characteristic - usually an organizational unit, a system, or an actor
- Notation: vertical (or horizontal) lanes with the partition name in a header box
- Every action lives in exactly one partition; edges freely cross lane borders - that is the hand-off
- Design payoff: partitions are candidate classes / services in the design phase

What is the primary purpose of partitions?

A) prettier diagrams B) group actions by actor / organizational unit

C) separate parallel processes D) show timing constraints

Object flow: data has a route too



- Object node (rectangle): a value at a point in the flow - 'order : Order' between producer and consumer
- Pins: tiny input/output object nodes glued to an action's border
- Data store: a central buffer that keeps a copy of EVERY token it receives (orders, invoices, patient records)

What triggers an action?

- Completion of the previous action: the default: the token arrives
- Availability of an object - the action starts when its input data is ready (object flow into the action)
- An external event - a user click, a message, a timer (UML has accept-event and time-event actions for these)

Class discussion - campus library borrow:

which trigger starts each action: scan card, check for fines, unlock shelf request, send due-date reminder?

2

Guided build: ATM withdrawal

We draw it together - requirements first, notation second.

Scenario: withdraw cash from an ATM

- User inserts card and enters PIN
- System validates PIN - 3 attempts allowed
- User selects withdrawal amount
- System checks account balance
- Sufficient funds: dispense cash and update balance
- Insufficient: show error message (and let the user try another amount)
- Print receipt - optional
- Return card

Step 1: identify components

- What are the main ACTIONS? (verb phrases)
- Where do we need DECISION nodes? What guards label their edges?
- Is there any CONCURRENCY? (honest answer for an ATM: ...)
- Who are the PARTITIONS? (customer / ATM / bank network)

Which nodes handle the 3 PIN attempts?
A) fork B) decision + merge (a guarded loop) C) join D) activity final

Stage 1: the skeleton (control flow only)

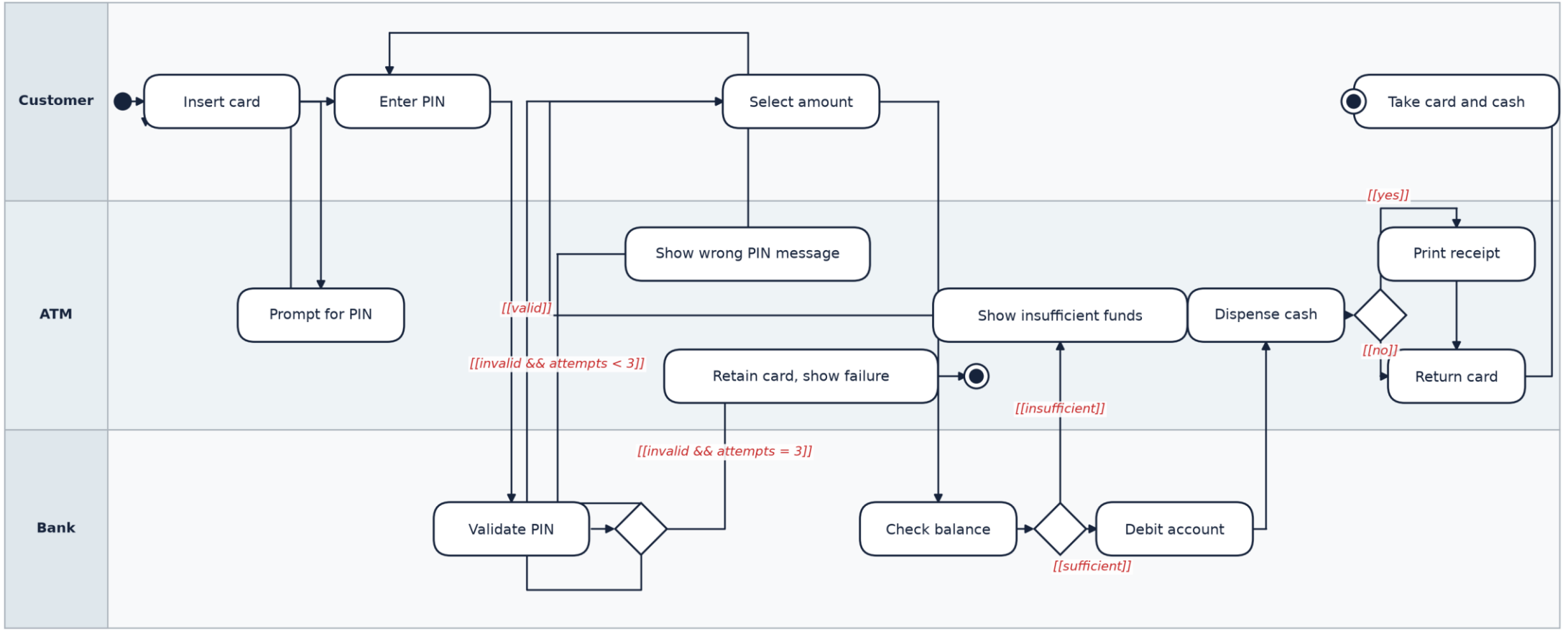
```
initial  ->  Insert card  ->  Prompt for PIN  ->  Enter PIN  ->  Validate PIN
         ->  Select amount  ->  Check balance  ->  Debit account
         ->  Dispense cash  ->  Return card  ->  Take card and cash  ->  final
```

- Happy path first: one straight line of actions, one initial node, one activity final
- Resist decisions until the spine works - same advice as writing the happy-path test first

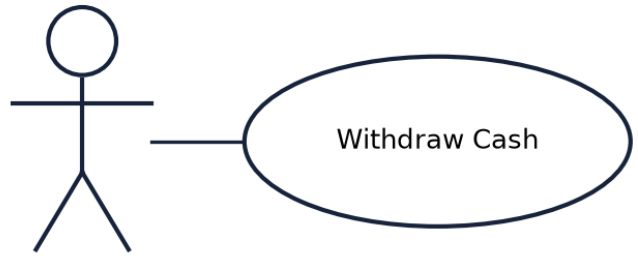
Stage 2: decisions, guards, loops

- After Validate PIN: decision with [valid], [invalid && attempts < 3] (loop back), [invalid && attempts = 3] (retain card → activity final)
- After Check balance: decision with [sufficient] and [insufficient] (loop back to Select amount)
- After Dispense cash: decision [print receipt?] yes / no - the no-edge skips one action and merges at Return card
- Every guard exclusive; every path ends at a final node - no dangling tokens

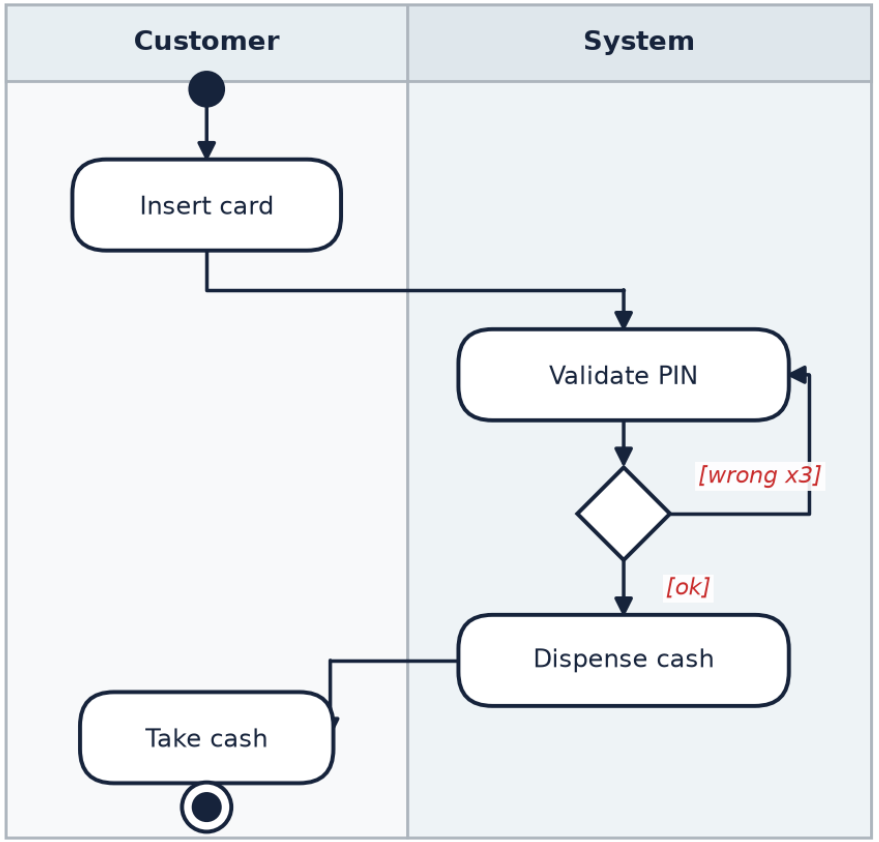
Stage 3: the whole activity, with partitions



From use case to activity diagram



realized by



Customer

Use case text (abbreviated):

1. Customer inserts card
2. System prompts for PIN
3. Customer enters PIN
4. System validates PIN
5. Customer selects amount
6. System dispenses cash

Extensions: 4a. wrong PIN -> retry (max 3)
6a. insufficient funds -> message

Same behaviour, three UML lenses

Activity diagram flow of actions, decisions, parallelism *“what is the workflow?”*

Sequence diagram messages between objects in time order *“who calls whom?” (client-server view)*

State machine states of one object + events *“what is this object's lifecycle?”*

- They complement, never replace, each other - a use case can be detailed by all three
- Activity diagrams hide message passing; sequence diagrams hide global workflow; state machines hide everything but one object

3

Where this lives in industry

The notation is UML; the ideas run production systems today.

Business process engines: BPMN

- BPMN 2.0 (OMG, 2011) is the business-facing sibling of UML activity diagrams: same DNA - gateways, forks, joins, pools/lanes
- Executable engines run these models as software: Camunda, Flowable and Activiti (open source) execute BPMN processes with service tasks, timers, human tasks
- Concept map: exclusive gateway ≈ decision+merge · parallel gateway ≈ fork+join · pool/lane ≈ partition
- Where you meet it: loan approval, order fulfillment, customer onboarding, insurance claims - anywhere humans + systems share a workflow

Cloud orchestrators speak fork/join fluently

AWS Step Functions	JSON state machines: Choice, Parallel, Map states
Azure Durable Functions	orchestrators; fan-out / fan-in is a named pattern
Apache Airflow	workflows as DAGs of tasks with dependencies
GitHub Actions	jobs + `needs:` graph; matrix jobs run in parallel
Temporal	durable workflows that survive crashes

- Same four concepts: guarded choice, fork, join, and data flowing between steps
- The diagram you learned today is the design language for all of these

And in your own code - already

- if / elif / else = a decision node with guards
- while / for = a loop: decision + back-edge
- threads, asyncio tasks, CompletableFuture, Promise.all = fork & join
- function parameters and return values = object flow
- classes / services with single responsibilities = partitions
- Every non-trivial program is an activity diagram that got compiled

Diagrams as code: PlantUML

```
@startuml
start
repeat
  :Fill registration form;
  :Validate input;
repeat while (input valid?) is (no)
:Create account;
:Send confirmation email;
:Show dashboard;
stop
@enduml
```

- Text in, diagram out - reviewable in pull requests, versionable in git
- Try it: plantuml.com/plantuml (paste & render)
- Drawing tools: draw.io, Lucidchart, Visual Paradigm; PlantUML for code-first teams
- Full sources for today's diagrams: code/activity_diagrams.puml

4

From diagram to running code

Every node type has an exact programming counterpart.

The mapping table

`action`	→ method / function (verb-phrase name)
`guard [c]`	→ if / elif condition c
`decision + back-edge`	→ while loop with bound
`fork / join`	→ concurrent tasks + wait-for-all (gather, allOf)
`object flow / pin`	→ parameter in, return value out
`data store`	→ database / repository
`partition`	→ class or service (single responsibility)
`activity final`	→ return / process exit

Guards & loops: the ATM PIN block in Python

```
while True:                                  # decision node after Validate PIN
    pin = customer.enter_pin()
    attempts += 1
    if bank.validate_pin(pin):               # guard [valid]
        break
    if attempts < MAX_PIN_ATTEMPTS:          # guard [invalid && attempts < 3]
        atm.show_wrong_pin_message()
    else:                                    # guard [invalid && attempts = 3]
        atm.retain_card_show_failure()
        return                               # activity final node
```

- code/atm_withdrawal.py - swimlanes as classes, actions as methods; 4 scenarios print full traces

Fork/join: async Python, measured

```
food    = asyncio.create_task(restaurant_prepares_food())     # FORK
courier = asyncio.create_task(dispatch_assigns_courier())     # FORK
food, courier = await asyncio.gather(food, courier)           # JOIN: waits for BOTH
```

Measured (code/order_workflow_async.py, kitchen 3 s, dispatch 2 s):

```
    sequential flows :  5.0 s   (= 3 + 2)
    fork / join      :  3.0 s   (= max(3, 2))
The join released the token when BOTH flows arrived - exactly the diagram's promise.
```

Same fork/join in Java

```
var food    = CompletableFuture.supplyAsync(OrderWorkflow::restaurantPreparesFood);   // FORK
var courier = CompletableFuture.supplyAsync(OrderWorkflow::dispatchAssignsCourier);   // FORK

CompletableFuture.allOf(food, courier).join();    // JOIN: proceeds when BOTH are done
```

Compiled & run with JDK 17 (code/OrderWorkflow.java):

```
join released after 3027 ms (sequential would be 5000 ms)
```

Partitions become classes

```
class PaymentService:                # lane: payments
    def confirm(self, order): ...
class KitchenService:                # lane: restaurant
    def prepare(self, order): ...
class DispatchService:               # lane: dispatch
    def assign_courier(self, order): ...

class DeliveryWorkflow:              # the activity itself (orchestrator)
    def deliver(self, order):
        if not self.payments.confirm(order):   # guard [payment ok]
            return                             # flow final
        ...
```

- code/order_services.py - analysis artifact → design sketch: lanes to classes, actions to methods

Best practices

DO

- verb-phrase action names
- one diagram, one concern
- swimlanes for ownership
- guards on every decision edge
- every path ends at a final node
- fork only where reality is parallel

DON'T

- mix abstraction levels
- draw 40-node monsters
- forget initial / final nodes
- leave dangling flows
- fork where things are sequential
- use activity final for one branch

Golden rule: if a newcomer cannot read your diagram in 2 minutes, it is too complex, split it.

The four classic mistakes

- 1. Decision/merge confused with fork/join → alternatives vs parallel: diamonds choose, bars multiply
- 2. Activity final where a flow final belongs → one branch's end kills all siblings
- 3. Missing or overlapping guards → undefined choice at run time (Bug Hunt #1)
- 4. Fork without synchronization → dangling flows, stuck tokens (Bug Hunt #2)

Five things to keep

- Activity diagrams model BEHAVIOUR: actions, guards, parallelism, ownership, data
- Diamonds choose (decision/merge); bars multiply and synchronize (fork/join)
- Guards must be exclusive and exhaustive - the model 'compiles' in a reviewer's head
- One use case → at least one activity diagram; extensions become guarded paths
- The diagram is the code's blueprint: guards→if, loops→while, fork/join→tasks+wait, lanes→classes

Resources & questions

- UML 2.5.1 specification (OMG, formal/2017-12-05) - Activities clause
- uml-diagrams.org - notation reference with examples
- Martin Fowler, UML Distilled (3rd ed.) · Craig Larman, Applying UML and Patterns (3rd ed.)
- PlantUML (plantuml.com) · draw.io · Lucidchart · Visual Paradigm
- Industry reading: Camunda / Flowable docs (BPMN), AWS Step Functions developer guide, Apache Airflow docs
- This lecture's assets: slides + code/ + diagrams/ + worksheet